

先進プロジェクト実験 I

IoT デバイス 最終報告

グループ番号：G6

2024 年 7 月 16 日提出

1. はじめに：IoT デバイス実験の目的

この IoT デバイス実験の目的は、IoT デバイスである Airoco と JetsonNano を用いて IoT に関する基本的な知識と技術を確認することである。さらに、ここで確認した知識や技術を実際に組み合わせ、「有用なサービスを作成する」という課題を通して実践し、習得することが本実験の目的である。

2. 4-7 回で制作した発展システム全体の説明

2.1 システムの目的

このシステムの最大の目的は、様々な制約によって通常生育が困難な植物を育てられるようにすることである。制約の例として、環境要因や法的要因などが挙げられる。このシステムを利用することで、対象とする環境を仮想的にコピーできるだけでなく、本来解決すべき制約の無視が可能ならば有用であると考えた。よって目的として定義した。

2.2 システムの構成

作成したシステムは、クライアント側と仮想サーバ側で構成が異なる。クライアント側の構成には UI の構築を目的として NodeRED を使った。また、エディタ URL にアクセスさせないために、Nginx によるリバースプロキシを使用した。

サーバサイドでは、flaskによるリクエスト処理とmysqlによるデータの管理を行った。また、flaskによるリクエスト処理と並行してAirocoの情報を取得してmysqlに登録するためのcrontabを設定した。flaskとcrontabはsupervisorによって制御した。

また、Node-RED、Nginx、Python、MysqlのそれぞれをDockerコンテナとして管理しており、クライアントサイドではNode-REDとNginxのDocker-compose、サーバサイドではPythonとMysqlのDocker-composeでそれぞれ実行している。

2.3 システムの実際の動き

システムは、大まかに準備、登録、実行に分けられるため、順に説明を行う。

2.3.1 システムの準備

システムを利用するには、はじめに案内のページに従って環境を用意する必要がある。案内ページはGitHubのPagesに公開されている。案内ではWSLの有効化、docker-desktopのインストール、リポジトリのダウンロード、docker-composeのコマンド実行方法、Node-REDのDashboardリンクが示されており、ユーザはPCにクライアント環境を作成することができる。これ以降のシステムの利用は、ローカルホスト上で行う。

2.3.2 システムへの登録

システムはクライアントサイドだけでは動作しない。システムを利用するには、ユーザ情報をサーバサイドに登録することを必要とする。

はじめに、2.3.1節でユーザがアクセスしたローカルホストの画面上で、ユーザが使用している任意のメールアドレスを入力する。この入力されたメールアドレスはflaskを通してmysqlのone_time_keysテーブルに登録される。表2.3.2.1にone_time_keysテーブルの仕様を示す。

表 2.3.2.1 ワンタイムキー送信時に更新されるデータテーブル one_time_keys の概要

属性	データ型	概要
id	int	ワンタイムキーごとに固有なidを示している。
email	varchar(100)	ユーザが登録したメールアドレスを示している。
one_time_key	varchar(6)	送信するワンタイムキーを示す※1。
expiration_time	datetime	期限切れになる時間（10分間）を示している。
created_at	timestamp	データが登録された時間を示している。
used	tinyint(1)	使用されたかどうか

※1 もし同行のemailが無効だった場合には、NULLが登録される。

表 2.3.2.1 で示したテーブルの概要にある、email には、一旦メールアドレスが格納されるが、このアドレスが無効だった場合にはワнтаイムキーが NULL となる。expiration_time は、created_at に 10 分を加算したデータが格納されている。used には、ワнтаイムキーが使用されたかどうか記録されており、デフォルトでは 0 が格納され、使用されると 1 が格納される。expiration_time と used は、一定時間間隔で値の評価が行われ、ワнтаイムキーが失効されたか使用されたことが分かったとデータが削除される。また、削除するために値を評価するために、イベントスケジューラーを使用している。

入力されたメールアドレスが有効だった場合、登録されたメールアドレスにサービス側からワнтаイムキーとワнтаイムキー認証ページへのリンクが記載されたメールの送信を行う。送信されたメールアドレスの内容の一部を、図 2.3.2.6 に示す。認証が完了すると、表 2.3.2.1 で示した属性 used が 1 になり、ワнтаイムキーが解放される。ただし、ワнтаイムキーが間違っている場合は、再入力を促すトーストを表示させる。

続いて、ユーザ情報の本登録画面へと遷移する。ここでユーザは、ユーザ名とパスワードの設定、設定したパスワードの再入力を行う。ここで入力された各情報は、flask を通して mysql の users テーブルに登録される。表 2.3.2.2 に users テーブルの概要を示す。

表 2.3.2.2 中の id は主キーであり、重複しない値となっている。username と password は、入力値をそのまま格納する。email はワнтаイムキーを送信したメールアドレスが登録される。このデータ格納が完了するとサインアップが完了する。

サインアップが完了すると個人アカウントの登録と同時に、flask を通し mysql の flower テーブルに初期マリーゴールドデータを格納する。表 2.3.2.3 に flower テーブルの概要を示す。表 2.3.2.3 の id は、users テーブルの id と対応している。flower_status には 21 種類あり、0 から 5 の良状態に加えて、1 から 5 の良状態を示す値 n に対して萎れ状態 n1、病気状態 n2、枯死状態 n3 の各悪状態のいずれかに分類される。各状態を示すイメージは (flower_status).png 形式で保持しており、図 2.3.2.4 に示す。また、図中の 999.png とは、何らかの原因で画像の表示がうまくいかないときに、デフォルトで表示される。

続いて、設定した username と password を入力してアプリケーションを開始する。サインイン画面とサインアップ画面にある hostIP には、サーバサイドのシステムを配信している PC の IP アドレスを指定しなければならない。

表 2.3.2.2 アカウント本登録時に更新されるデータテーブル users の概要

属性	データ型	概要
id	int	ユーザごとに固有な id を示している。
username	varchar(50)	ユーザが任意で決定する名前
password	varchar(255)	ユーザが任意で決定するパスワード
email	varchar(100)	ユーザが登録したメールアドレスを示している。
created_at	timestamp	データが登録された時間を示している。

表 2.3.2.3 アカウント本登録時に更新されるデータテーブル users の概要

属性	データ型	概要
id	int	ユーザごとに固有な id を示している.
flower_status	int	花の状態を数値で示す.
water	int	水分量を示す. 0 から 100 で表現される.
fertilizer	int	肥料量を示す. 0 から 100 で表現される.
live_time	int	次の良状態に向かうための数値.
droop_time	int	今の状態から病気状態に向かう数値.
wither_time	int	今の状態から萎れ状態に向かう数値.
death_time	int	今の状態から枯死状態に向かう数値.
in_the_stauts_time	int	その状態で経過した時間(min)

また, *_time 属性は mysql の内部で更新され, デフォルトは 0 である. 各値を評価・更新するプロシージャを毎分実行するように設定しており, 各値の評価には Airoco が収集した最新のデータを使用する. crontab が, データを取得して mysql に登録する処理をまとめた python ファイルを毎分実行し, sensor_table_R3_401 テーブルに格納する. sensor_table_R3_401 テーブルの概要を表 2.3.2.5 に示す.

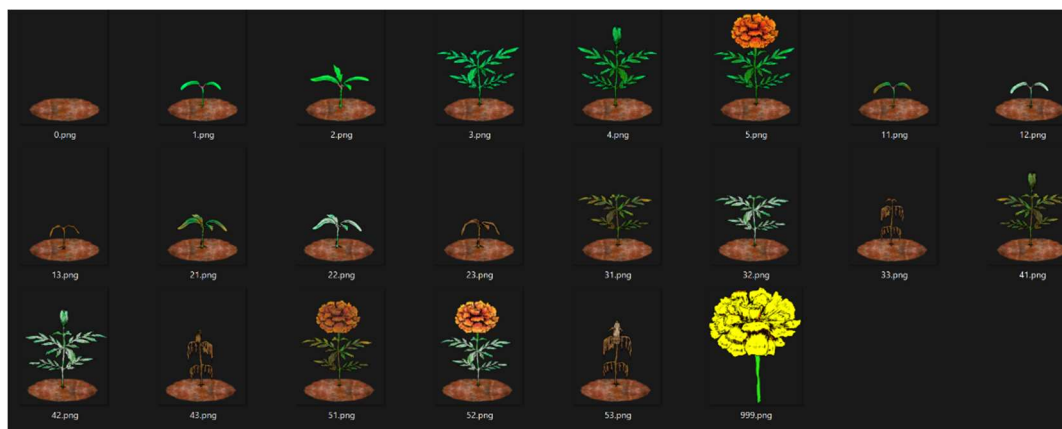


図 2.3.2.4 Node-RED コンテナ内で保持しているイメージ一覧

表 2.3.2.5 アカウント本登録時に更新されるデータテーブル users の概要

属性	データ型	概要
id	int	登録されたデータごとに割り振る一意な id
time_stamp	datetime	登録時に取得するタイムスタンプ
co2	float	Airoco の CO2[ppm]が格納される
humidity	float	Airoco の湿度[%]が格納される
temperature	float	Airoco の気温[°C]が格納される

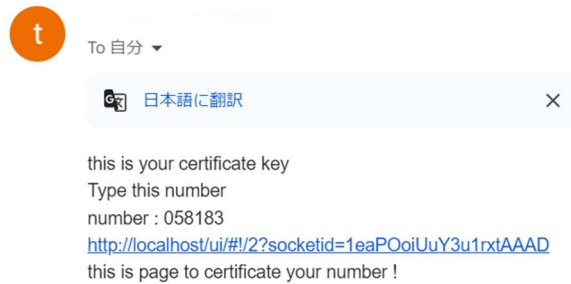


図 2.3.2.6 認証のために送信されるメールの中身

表 2.3.2.5 のテーブルでは, time_stamp の値が現在時刻と比較して 1 週間以上差があった場合に該当するすべての行を削除する. この操作は, mysql 内で作成したイベントによって実行され, 24 時間ごとに起動するように設定している.

2.3.2 節で使用したメールアドレス入力画面, ワンタイムキー認証画面, アカウント作成画面, サインイン画面を, 図 2.3.2.7 にそれぞれ示す.

The figure consists of four screenshots of web forms arranged in a 2x2 grid. The top-left form is titled 'please your information' and has input fields for 'email *' and 'hostIP', with 'SUBMIT' and 'CANCEL' buttons at the bottom. The top-right form is titled 'type here your certificate number' and has a 'certificate number *' input field with 'SUBMIT' and 'CANCEL' buttons. The bottom-left form is titled 'please infomation to resister' (note the typo) and has input fields for 'username * Name', 'password *', and '[re]password *', with 'SUBMIT' and 'CANCEL' buttons. The bottom-right form is also titled 'please infomation to resister' and has the same fields and buttons as the bottom-left form.

図 2.3.3.7

- (左上) はじめにメールアドレスを入力する画面
- (右上) 入力されたメールアドレスに送信されたワンタイムキーを入力する画面
- (左下) 本登録のためにユーザ名とパスワードを設定する画面
- (右下) 本登録したユーザ名とパスワードを入力してサインインを完了する画面

2.3.3 システムの実行

2.3.2 節までが完了するとアカウントとアカウントに紐づくマリーゴールドの情報を生成される。ここで、サインインタブを選択して登録したユーザ名とパスワードを入力することで、各ユーザに紐づくマリーゴールドの情報に基づき DashBoard の表示をさせることができる。

DashBoard では、機能ごとにウィンドウを分けて画面を表示する。図 2.3.3.1 では、ユーザのアプリケーション画面を示す。図内の各ウィンドウは、Airoco データを表示する左部、状態をイラストで表現する中央部、詳細情報を表示する右上部、マリーゴールドの水分・栄養状態を示す右中央部、flower テーブルの*_time で表される各値の表示を行う右下部に分かれる。

左部ウィンドウと右中央部ウィンドウの「<」と「>」はボタンになっておりクリックすることで、左部ウィンドウでは co2 ウィンドウ・humidity ウィンドウ・temperature ウィンドウへと遷移する。一方、右中央部ウィンドウでは fertilizer ウィンドウ・water ウィンドウへと遷移する。これらの遷移において画面のレイアウトは変化せず、ウィンドウの内容のみが遷移する。実際に、両ウィンドウのボタンをクリックした後の画面を図 2.3.3.2 に示す。図 2.3.3.1 と図 2.3.3.2 の右中央部ウィンドウでは、fertilizer ウィンドウと water ウィンドウで表示されるボタンを「>」・「<」として分けている。この違いには、ウィンドウの行き来を視覚的にわかりやすくするねらいがある。

図 2.3.3.1 の右上のウィンドウではアイコンが五つ並んでおり、それぞれボタンやテキスト表示として機能する。各ボタンは、左からユーザ情報表示、天気情報表示、背景色変更の機能を持つボタンがある。続く四つ目のボタンと五つ目のボタンでは、tooltip 機能を用いてカーソルをホバーさせると特定のデータを表示させる。

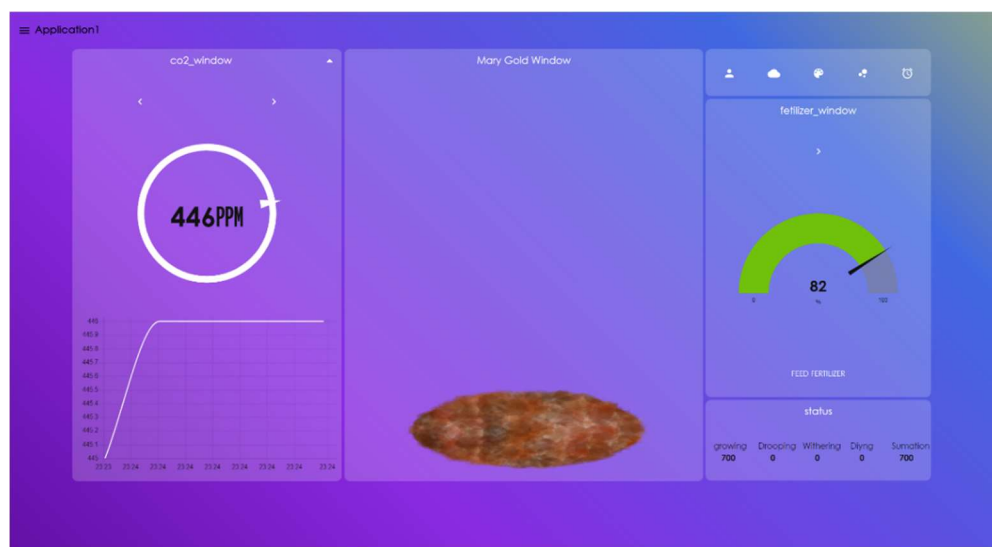


図 2.3.3.1 システムのアプリケーション画面

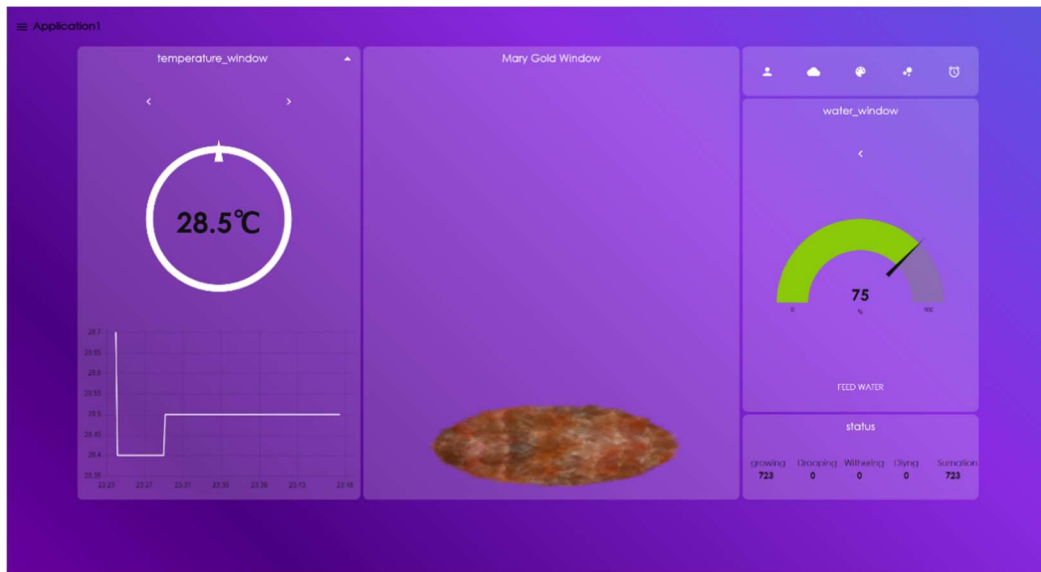


図 2.3.3.2 遷移ボタンを押すことで内容が変わった各ウィンドウ

図 2.3.3.1 の右上のウィンドウではアイコンが五つ並んでおり、それぞれボタンやテキスト表示として機能する。各ボタンは、左からユーザ情報表示、天気情報表示、背景色変更の機能を持つボタンがある。続く四つ目のボタンと五つ目のボタンでは、tooltip 機能を用いてカーソルをホバーさせると特定のデータを表示させる。

前述した五つのボタンに対するアクションの結果を図 2.3.3.3 から図 2.3.3.7 に示す。図 2.3.3.3 と図 2.3.3.4 で示したウィンドウは、ボタンをクリックした結果遷移するタブに表示されるウィンドウである。左列のラベルに対応するデータを表示する。図 2.3.3.3 には、users テーブルに格納した値と対応したデータを表示させる。また図 2.3.3.4 には気象庁から取得した気象情報に基づき、名古屋周辺におけるおおよそその一日の天気情報を表示する。また、元のシステム画面に戻るには「<」ボタンをクリックすればよく、タブの遷移により図 2.3.3.1 の画面まで戻ることができる。

図 2.3.3.5 では、クリックするごとに背景色が変化することを示す。背景色は完全にランダムに 4 色が選択され、線形グラデーションとしてセットされる。

図 2.3.3.6 と図 2.3.3.7 では、クリックしても変化は起きないが、カーソルをホバーさせることで情報を表示する。前者では、植物の状態である flower_status の 0 から 4 までの値 n に対応したレベルが「Lv.(n+1)」表示される。また、flower_status が 5 になり成長がストップすると、表示が great に切り替わる。後者では、最終更新時刻が表示される。最終更新時刻は、画面上の各要素が更新されるたびに増加する。

続いて、右中央部のウィンドウについて説明する。前述したマリーゴールドの水分・栄養状態を確認できるだけでなく、FEED ボタンをクリックして 100% に戻すことができる。水分は 20 分に 1%、肥料は 30 分に 1% ずつ減少するため、定期的に水やりと肥料を与えることが必要とされる。操作の結果は flask を通して flower テーブルへ反映される。

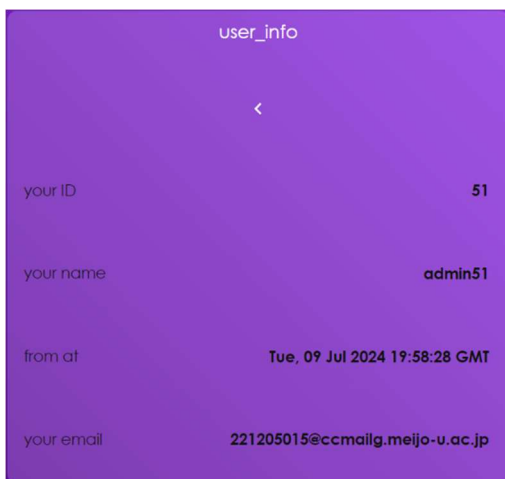


図 2.3.3.3 一つ目のボタンから遷移するタブ



図 2.3.3.4 二つ目のボタンから遷移するタブ

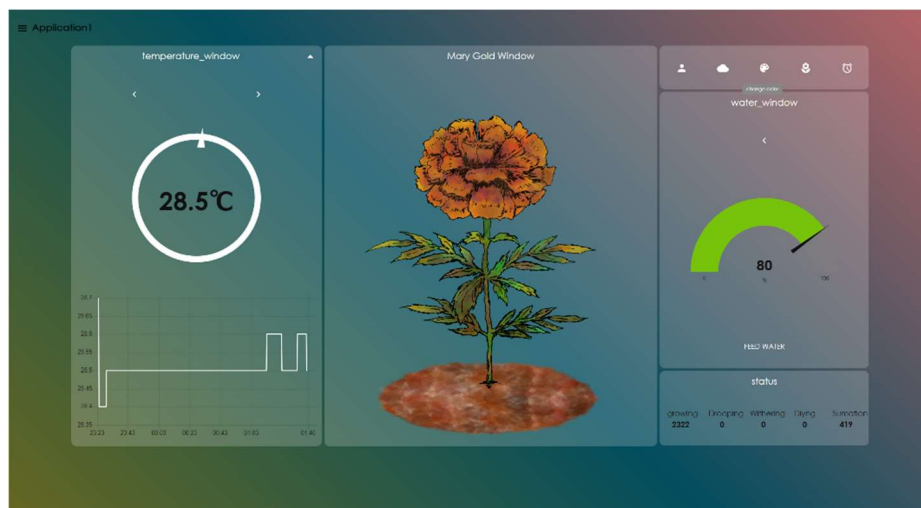


図 2.3.3.5 三つ目のボタンにより背景色が変化した

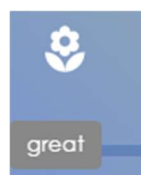


図 2.3.3.6 四つ目のボタンをホバーすると表示される flower_status の状態表示

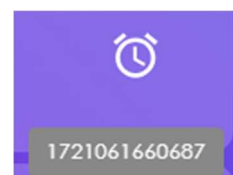


図 2.3.3.7 五つ目のボタンをホバーすると表示される最終更新時刻

また、図 2.3.3.2 や図 2.3.3.5 の中央部のイラストは、2.3.2 節の図 2.3.2.4 に示す画像の一部であり、Nginx によって静的コンテンツとしてローカルで配信する画像を Node-RED の template ノード内で参照して表示を行う。実際に図 2.3.3.2 では 0.png が表示され、図 2.3.3.5 では 51.png が表示されている。仮に、画像紛失などの原因によってデータの表示が行われない場合は 999.png が表示される。そもそもローカルに画像が配信されていなかった場合には、alt タグによって代替テキストが表示されるようになっている。

3. 発展システムで自分が担当した部分の説明

3.1 全体の中のどこを担当したか

私は主に、システム全体の通信の基盤と、通信に関わるアプリケーションの機能を担当しました。例として、コンテナによるネットワークの構築, flask による通信の受け渡し, Node-RED の request ノードによる flask コンテナへの接続が挙げられます。

そして、Mysql コンテナ内で作成した各テーブル, 各イベント, 設計を除くプロシージャの実装も私が担当しました。

また、Nginx による静的コンテンツの設定やリバースプロキシの設定, Python コンテナ内で動作する crontab の設計も私が担当しました。

最後に、UI の最終調整とシステム全体の動作テストを私が行いました。

3.2 担当部分の動きの説明

システム全体の通信の基盤とは、コンテナ同士の通信のことを指します。システム自体は docker-compose で各コンテナを起動させており、各コンテナがひとつのサービスとして協調して動作する。

図 3.2.1 に、コンテナがどのように稼働しているかを示す。図 3.2.1 内の各コンテナは大まかにクライアントサイドとサーバサイドに分離している。また、両者の通信は LAN 内において IP アドレスを指定して行われる。この IP アドレスは、システム実行時と登録時に入力する hostIP と紐づいている。

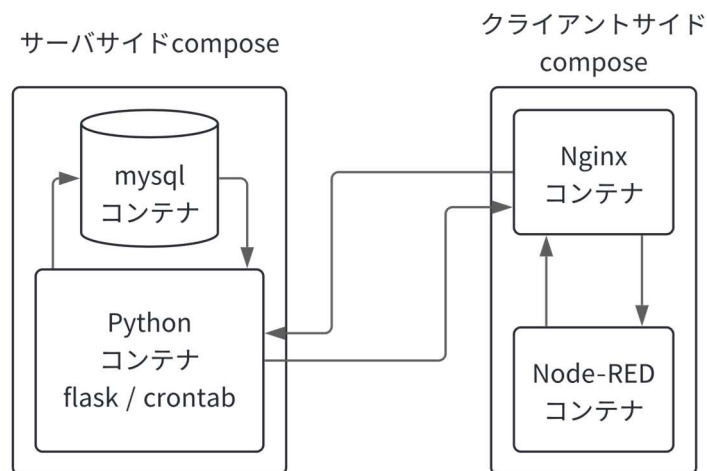


図 3.2.1 コンテナ間のデータフロー

flask では、mysql と node-RED 間の通信における仲介を行うことで、クエリチェック、更新・追加・取得といった基本的な機能を果たす。例として、サインイン時に入力されたデータに合致する行を users テーブルから検索したり、Node-RED で水分か肥料の FEED ボタンを押されたときに最新の情報に更新したりする。また、クエリが欠損していると、mysql に対する操作はせずに失敗したことを示すメッセージを返す。

mysql で動作する各イベントとして、表 3.2.2 にまとめる。各イベントではインターバルが設けられており、指定された時間が過ぎる度に同じイベントを実行する。これは、サーバが稼働している限り常にデータが更新されることを意味するので、ユーザに継続的なサインインとデータの確認を要求することにつながる。

表 3.2.2 Mysql コンテナ内で定義されているイベントの概要

イベント名	間隔	説明
decrease_fertilizer	30min	30 分毎に 1%肥料が減少する
decrease_water	20min	20 分毎に 1%水分が減少する
delete_expired_records	1min	1 分毎に、無効なワンタイムキーを削除
delete_old_records	1day	1 日毎に、一週間以上前の Airoco データを削除
update_flower_status_event	1min	1 分毎に flower_status と*_time を更新

また、静的コンテンツの配信や、リバースプロキシの設定は nginx の default.conf で設定している。実際に、Dashboard は <http://localhost/ui> として配信しており、サーバサイドの flask は <http://localhost/api> としてアクセスする。また、画像については、node-red 内の画像を格納しているディレクトリを <http://localhost/> として配信しており、ファイル名を指定して表示を行う。

3.3 特に工夫した部分

特に工夫した部分は、実際にサービスとして配信できるような形態にしたことである。実際に、Docker-compose によるコンテナのデプロイは AWS の ECS / ECR や Heroku などでサポートされている。

また、ユーザアカウント制は特にこだわって工夫した点である。ユーザごとに決められたユーザ名とパスワード、花の状態などを管理して、ユーザごとに異なる DashBoard を提示することができれば、より有用であると考えたからである。

最後に、UI の背景やコンポーネントの微調整にも工夫を凝らした。DashBoard にアクセスしたときに汚い UI である場合や、単調でつまらない画面である場合だと、サービスとして長続きしないと考えたためである。また、テーマとして「仮想的に育成する」ことを掲げているので、グラデーションな背景によって電腦感を演出したいという想いも込められている。

4. プレゼンに関して

4.1 担当した部分

私が担当したのは、プレゼン本番のサインアップからサインインまでの説明である。
また、スライドに最終調整で変更された UI の反映を行った。

4.2 プレゼンの自己評価

プレゼンでは、伝えたいことをすべて説明することができた。しかし、本番では少し緊張してしまったため、早口で話してしまったことに加え、想定外のことが PC 上で起こってしまったこともあり、実際に聞いている人に伝えられたかどうか自信がない。

さらに、質問の内容を自分たちだけが正確につかむことができず、質疑応答の時間を無駄にしまったこともあり、聞き手にとって良くない発表にしまったかもしれないと感じた。

以上から、プレゼンの自己評価としてはかなり低めであり、準備が足りていなかったことをきわめて強く痛感した。

5. まとめ

5.1 自己評価

全体を通して、グループに貢献できた箇所はあるのではないかと考えている。しかし、慣れない環境での実行や、共同作業のためのファイル共有や GitHub を使った CI/CD の設定などが適切に行えなかったため、グループに迷惑をかけてしまったとも感じている。

個人的な目標として、データベースをうまく活用してシステムを構築することを掲げていたが、これは達成できたと考えている。また、今まで大学で学んだ flask やデータベースの知識だけでなく、新たに得た Docker やプロキシなどの知識を組み合わせることができた。

以上のから、全体を通した自己評価としては高めであり、今後はグループで活動する上で必要な知識についても収集した上でプロジェクトの提案ができればより高く自己評価ができたと感じている。

5.2 感想

今回の実験を通して、新たに使用するソフトウェアをチームで共有する困難さや、進捗管理ツールの重要性を痛感した。

また、論文や企業の研究データといった信頼できるデータをもとにしてシステムを作成することで、システムの信頼性や有用性を保証する必要性についても再確認した。

今回「信頼できる情報」がつけられなかった原因として、海外のデータを参照しなかったことや、二酸化炭素濃度・湿度・気温を同時に扱っているデータを探していたことが挙げられるのではないかと考えた。例えば、気温、湿度、二酸化炭素濃度をそれぞれ別に収集しているデータを結合させることで、花の状態をよりリアルに変化させることもできたはずである。

今後、自分でシステムや Web サービスを構築する際には、今回学んだことを活かして次につなげていこうと考えている。

5.3 その他

作成したクライアントサイドの github リポジトリ

<https://github.com/UmaShiny/iot6-yml>